universität innsbruck

Review on Machine Learning

LV-703330: Machine Learning for Interactive Systems

Assoz.Prof. Antonio Rodríguez-Sánchez, PhD.

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

Machine Learning

- A *training set* is used to tune the parameters of and adaptive model

$$\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$$

- The precise form of $\mathbf{y}(\mathbf{x})$ is determined during the training or learning phase.
- We can determine the identity of new input: the *test set*.
- The ability to categorize correctly new examples from the training set is known as generalization.

Machine Learning

- A *training set* is used to tune the parameters of and adaptive model

$$\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$$

- The categories are known in advance: a target vector $\mathbf{t}$ which represents the identity of the corresponding input.
- The result of running a machine learning algorithm can be expressed as a function $\mathbf{y}(\mathbf{x})$ which generates an output and generates an output vector $\mathbf{y}$

More terms

- Preprocessing or *feature extraction*.
  - Original input is transformed into a new space of variables.
- Supervised learning.
  - Training data comprises examples of input and target vectors.
  - Regression: Output consists of one or more continuous variables.
- Unsupervised learning.
  - Input vectors without target values.
  - Clustering: Groups of similar examples.
  - Density estimation: Determine the distribution of data within input space

Chain rule of Probability

The Rules of Probability

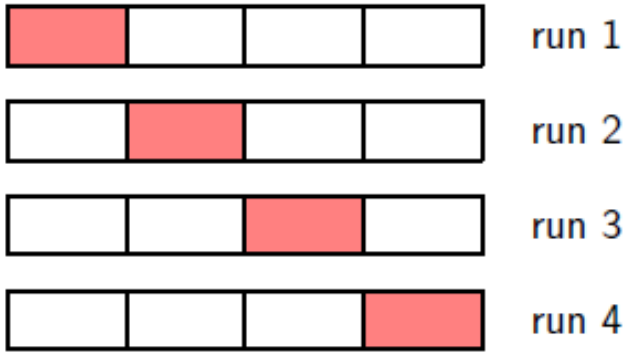
sum rule $$p(X) = \sum_{Y} p(X, Y)$$

product rule $$p(X, Y) = p(Y|X)p(X).$$

$$P(\mathrm{x}^{(1)}, \ldots, \mathrm{x}^{(n)}) = P(\mathrm{x}^{(1)}) \Pi_{i=2}^{n} P(\mathrm{x}^{(i)} \mid \mathrm{x}^{(1)}, \ldots, \mathrm{x}^{(i-1)})$$

Cross-validation

- A proportion $(S-1)/S$ of the available data is to be used for training while making use of all of the data to assess performance



Inference

- Suppose we have an input vector $\mathbf{x}$ together with a corresponding vector $\mathbf{t}$ of target variables, and our goal is to predict $\mathbf{t}$ given a new value for $\mathbf{x}$
  - For regression problems, $\mathbf{t}$ will comprise continuous variables
  - For classification problems $\mathbf{t}$ will represent class labels
  - We want to determine $p(\mathbf{x},\mathbf{t})$ from a set of training data: *inference*
    - This is a very hard problem
  - In the end we need to determine some value for $\mathbf{t}$

Inference

- Suppose we have an input vector $\mathbf{x}$ together with a corresponding vector $\mathbf{t}$ of target variables, and our goal is to predict $\mathbf{t}$ given a new value for $\mathbf{x}$
  - We want to determine $p(\mathbf{x},\mathbf{t})$ from a set of training data: *inference*
    - This is a very hard problem
  - In the end we need to determine some value for $\mathbf{t}$
  - E.g. $t=0$ (class C_1): presence of cancer; $t=1$ (class C_2): absence of cancer
  - We have to choose the *optimal* decision

Inference

- Suppose we have an input vector $\mathbf{x}$ together with a corresponding vector $\mathbf{t}$ of target variables, and our goal is to predict $\mathbf{t}$ given a new value for $\mathbf{x}$
  - We want to determine $p(\mathbf{x},\mathbf{t})$ from a set of training data: *inference*
    - This is a very hard problem
  - In the end we need to determine some value for $\mathbf{t}$
  - E.g. $t=0$ (class C_1): presence of cancer; $t=1$ (class C_2): absence of cancer
  - We have to choose the *optimal* decision
  - Now, we need to evaluate new patient

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}.$$

$p(C_1)$ *represents* the probability that a person has cancer, before we take the X-ray measurement

$p(C_1|\mathbf{x})$ is the corresponding probability, in light of the information contained in the X-ray

Inference

- Suppose we have an input vector $\mathbf{x}$ together with a corresponding vector $\mathbf{t}$ of target variables, and our goal is to edict $\mathbf{t}$ given a new value for $\mathbf{x}$

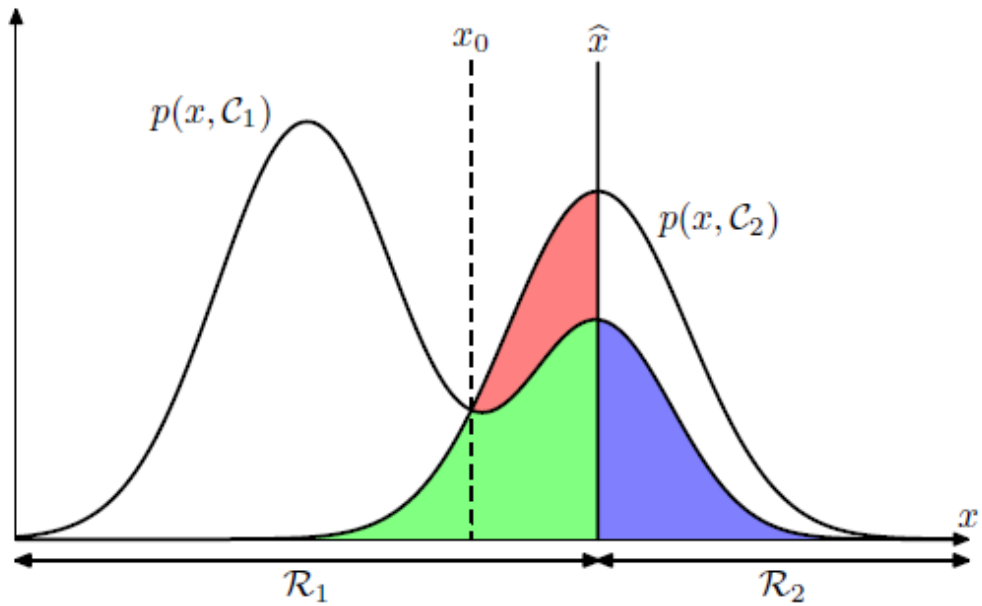
$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}.$$

$p(C_1)$ *represents* the probability that a person has cancer, before we take the X-ray measurement

$p(C_1|\mathbf{x})$ is the corresponding probability, light of

  - We want to minimize the

$$\begin{aligned} p(\text{mistake}) &= p(\mathbf{x} \in \mathcal{R}_1, \mathcal{C}_2) + p(\mathbf{x} \in \mathcal{R}_2, \mathcal{C}_1) \\ &= \int_{\mathcal{R}_1} p(\mathbf{x}, \mathcal{C}_2)\,\mathrm{d}\mathbf{x} + \int_{\mathcal{R}_2} p(\mathbf{x}, \mathcal{C}_1)\,\mathrm{d}\mathbf{x} \end{aligned}$$



Inference

- The cost (or loss) function
  - E.g. $t=0$ (class C_1): presence of cancer; $t=1$ (class C_2): absence of cancer
  - Consequences of diagnosing as healthy a patient with cancer have much more dramatical consequences than the opposite.

Inference

- The cost (or loss) function
  - E.g. $t=0$ (class C_1): presence of cancer; $t=1$ (class C_2): absence of cancer
  - Consequences of diagnosing as healthy a patient with cancer have much more dramatical consequences than the opposite.
  - Loss matrix

$$\begin{array}{c} \\ \text{cancer} \\ \text{normal} \end{array} \begin{array}{c} \begin{array}{cc} \text{cancer} & \text{normal} \end{array} \\ \begin{pmatrix} 0 & 1000 \\ 1 & 0 \end{pmatrix} \end{array}$$

  - The loss function depends on the true class: it is unknown
  - So we seek to minimize the average loss: Choose the regions R_j that minimize the expected loss

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj} p(\mathbf{x}, \mathcal{C}_k)\, \mathrm{d}\mathbf{x}. \sim \sum_k L_{kj} p(\mathcal{C}_k|\mathbf{x})$$

Inference

- Three approaches
  - Generative models: Determine $p(\mathbf{x}|C_k)$ and $p(C_k)$ and apply Bayes theorem to get the posterior and use decision theory

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} \qquad p(\mathbf{x}) = \sum_k p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k).$$

  - Discriminative models: Determine the posterior class probabilities $p(C_k|\mathbf{x})$ and then apply decision theory

Inference

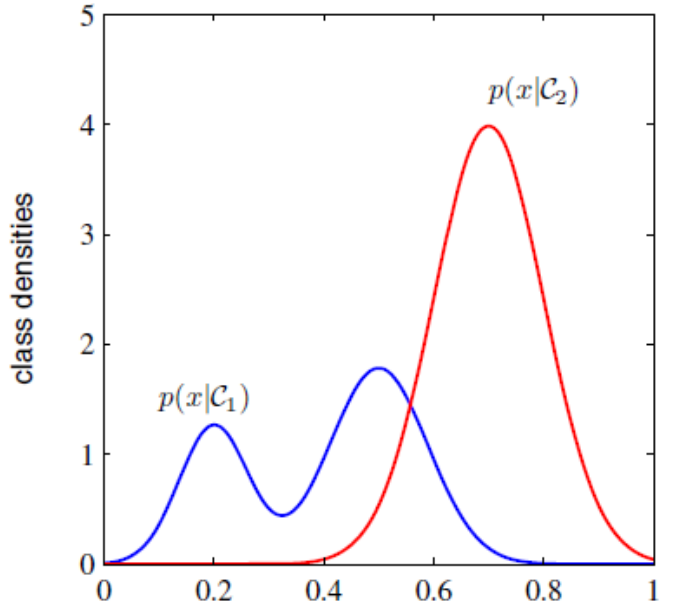
- Three approaches
  - Generative models: Determine $p(\mathbf{x}|C_k)$ and $p(C_k)$ and apply Bayes theorem to get the posterior and use decision theory

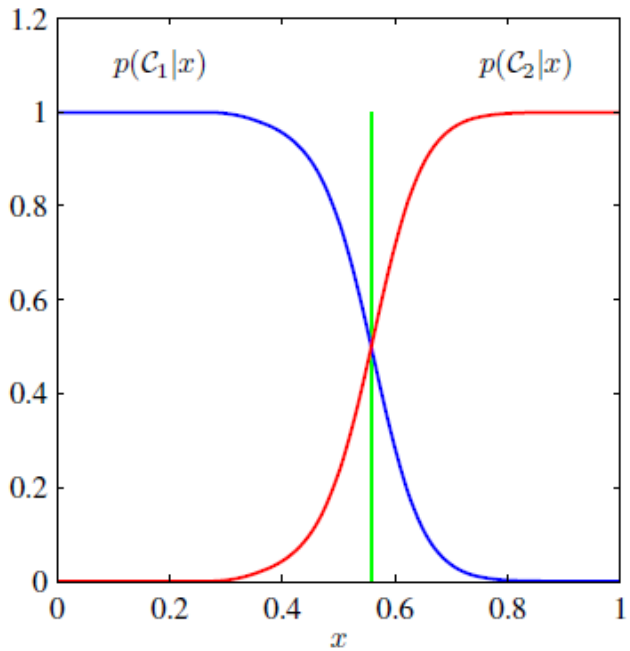
$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} \qquad p(\mathbf{x}) = \sum_k p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k).$$

  - Discriminative models: Determine the posterior class probabilities $p(C_k|\mathbf{x})$ and then apply decision theory
  - Discriminant function: A function $f(x)$ that maps directly x onto a class label
    - E.g. in a 2 class label, $f=0$ or $f=1$
    - Probabilities play no role

Inference

- Three approaches
  - Generative models
  - Discriminative models
  - Discriminant function





Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

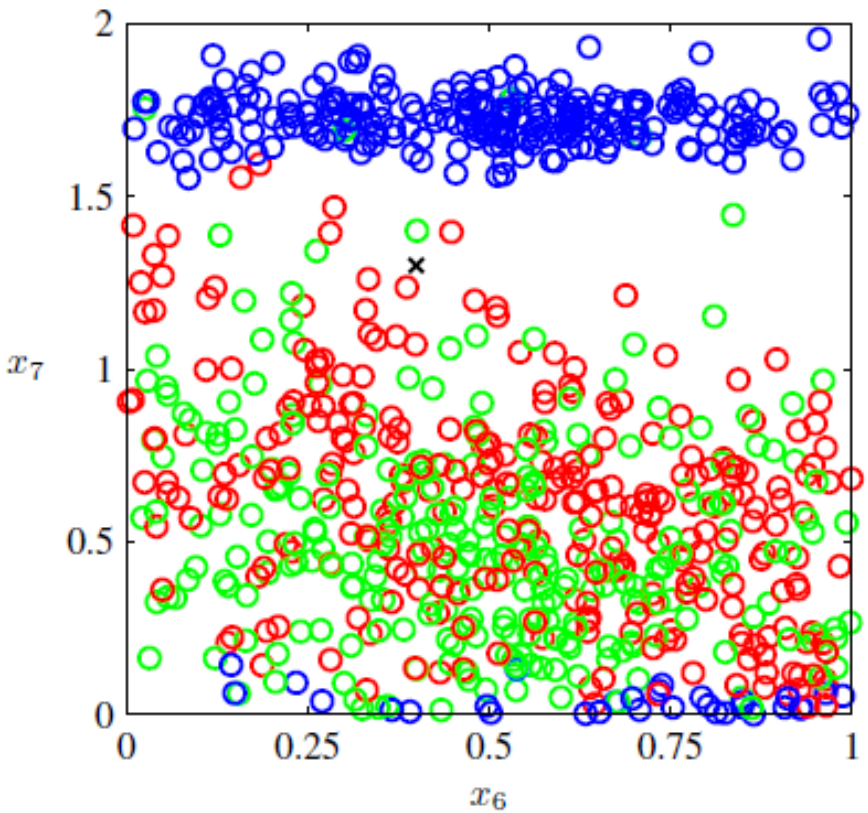
Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

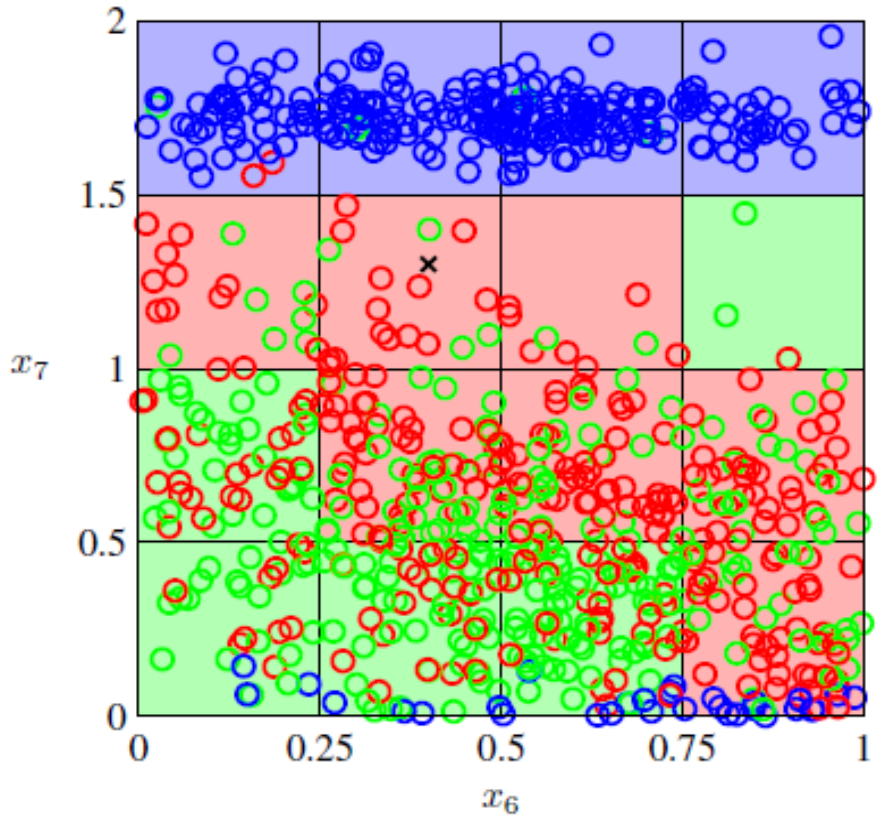
The curse of dimensionality

- We usually have to deal with spaces of high dimensionality: many input variables
- Consider the case where we have 100 points and each data point is a 12-dimensional input vector



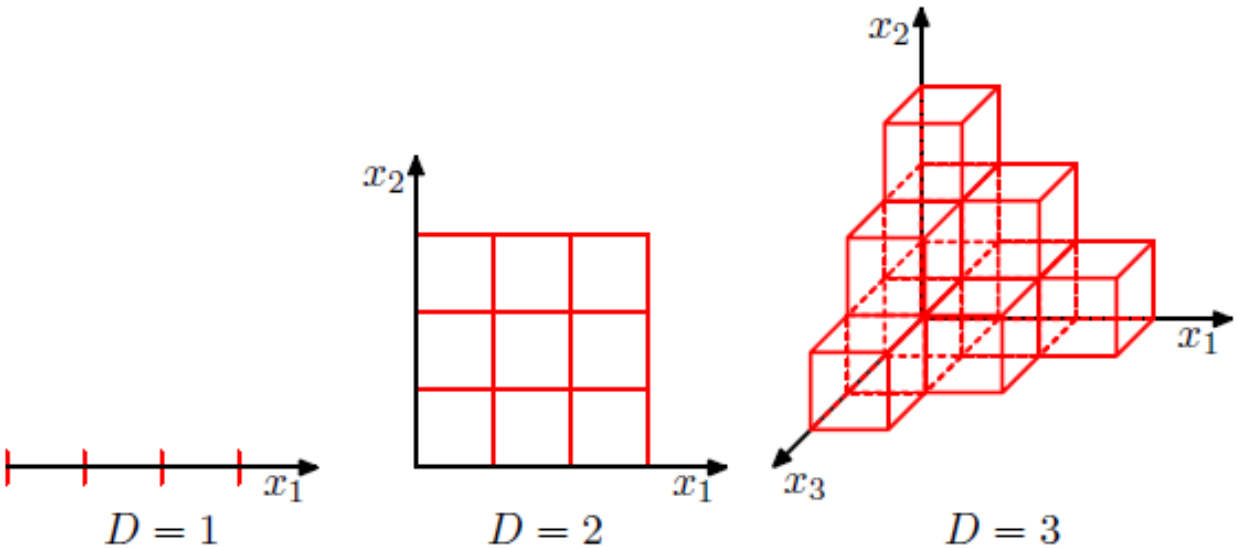
The curse of dimensionality

- We usually have to deal with spaces of high dimensionality: many input variables
- Consider the case where we have 100 points and each data point is a 12-dimensional input vector
- How do we solve the class for x
  - Divide into a grid



The curse of dimensionality

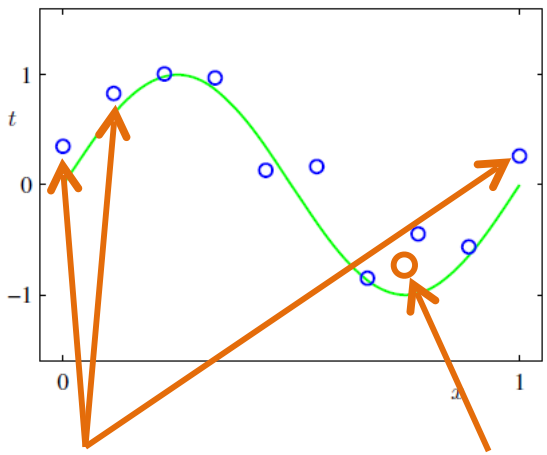
- We usually have to deal with spaces of high dimensionality: many input variables
- Consider the case where we have 100 points and each data point is a 12-dimensional input vector
- How do we solve the class for x
  - Divide into a grid
  - Take this approach for the 12D vector



The curse of dimensionality

- We usually have to deal with spaces of high dimensionality: many input variables
- Let's look at the math: Use the example of the polynomial curve fitting
  - But let's take it to a higher dimension, e.g. 3

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} x_i x_j + \sum_{i=1}^{D}\sum_{j=1}^{D}\sum_{k=1}^{D} w_{ijk} x_i x_j x_k.$$

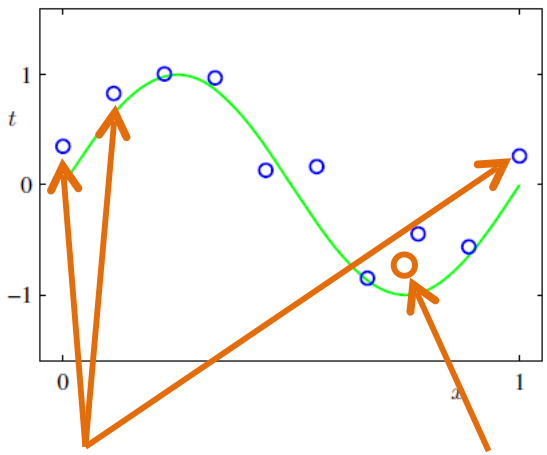


N observations x = $(x_1, \ldots, x_N)^T$ *t* value?

The curse of dimensionality

- We usually have to deal with spaces of high dimensionality: many input variables
- Let's look at the math: Use the example of the polynomial curve fitting
  - But let's take it to a higher dimension, e.g. 3

$$y(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} x_i x_j + \sum_{i=1}^{D}\sum_{j=1}^{D}\sum_{k=1}^{D} w_{ijk} x_i x_j x_k.$$



N observations x = $(x_1, \ldots, x_N)^T$

t value?

  - As D increases, the number of independent coefficients grows to D^3
  - So, for a polynomial of order M, we need D^M coefficients

The curse of dimensionality

- We usually have to deal with spaces of high dimensionality: many input variables
- We can solve this considering two properties of data
  - the directions over which important variations in the target variables occur may be confined to a smaller dimension
  - real data will typically exhibit some smoothness properties, and so we can exploit local interpolation-like techniques

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

Information theory

- Quantifies how much information is present in a signal

Information theory

- Quantifies how much information is present in a signal
- Learning that an unlikely event has occurred is more informative than learning a likely event
  - Likely events should have low information content
  - Less likely events should have higher information content
  - Independent events should have additive information

Information theory

- Quantifies how much information is present in a signal
- Self-information of an event $\mathrm{x} = x$ is

$$h(x) = -\log_2 p(x)$$

  - Low probability events x correspond to high information content
  - Units are bits
  - Can be natural base as well, in which case units are called nats

Information theory

- Quantifies how much information is present in a signal
- Shannon entropy

$$\mathrm{H}[x] = -\sum_x p(x) \log_2 p(x) \qquad\qquad h(x) = -\log_2 p(x)$$

  - Amount of uncertainty in an entire probability distribution
  - The expected amount of information in an event drawn from that distribution

Information theory

- Quantifies how much information is present in a signal
- Shannon entropy

$$\mathrm{H}[x] = -\sum_x p(x) \log_2 p(x) \qquad h(x) = -\log_2 p(x)$$

  – Example: Consider a random variable x having 8 possible states, each equally likely

  – In order to communicate value x to a receiver, we need 3 bits

$$\mathrm{H}[x] = -8 \times \frac{1}{8} \log_2 \frac{1}{8} = 3 \text{ bits.}$$

Information theory

- Quantifies how much information is present in a signal
- Shannon entropy

$$\mathrm{H}[x] = -\sum_x p(x) \log_2 p(x) \qquad\qquad h(x) = -\log_2 p(x)$$

  – Example: Consider a random variable x having 8 possible states, now probabilities $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}\right)$
  – If we use shorter codes for more probable events and longer for less probable events, on average

$$\mathrm{H}[x] = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{4}\log_2\frac{1}{4} - \frac{1}{8}\log_2\frac{1}{8} - \frac{1}{16}\log_2\frac{1}{16} - \frac{4}{64}\log_2\frac{1}{64} = 2 \text{ bits}$$

Information theory

- Quantifies how much information is present in a signal
- Shannon entropy

$$\mathrm{H}[x] = -\sum_x p(x)\log_2 p(x) \qquad\qquad h(x) = -\log_2 p(x)$$

  – Example: Consider a random variable x having 8 possible states, now probabilities $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}\right)$
  – If we use shorter codes for more probable events and longer for less probable events, on average

$$\text{average code length} = \frac{1}{2}\times 1 + \frac{1}{4}\times 2 + \frac{1}{8}\times 3 + \frac{1}{16}\times 4 + 4\times\frac{1}{64}\times 6 = 2\text{ bits}$$

Information theory

- Relative entropy and mutual information
- When we have two separate probability distributions $p(\mathrm{x})$ and $q(\mathrm{x})$ over the same random variable x, we can measure how different they are

Information theory

- Relative entropy and mutual information
- When we have two separate probability distributions $p(\mathrm{x})$ and $q(\mathrm{x})$ over the same random variable x, we can measure how different they are

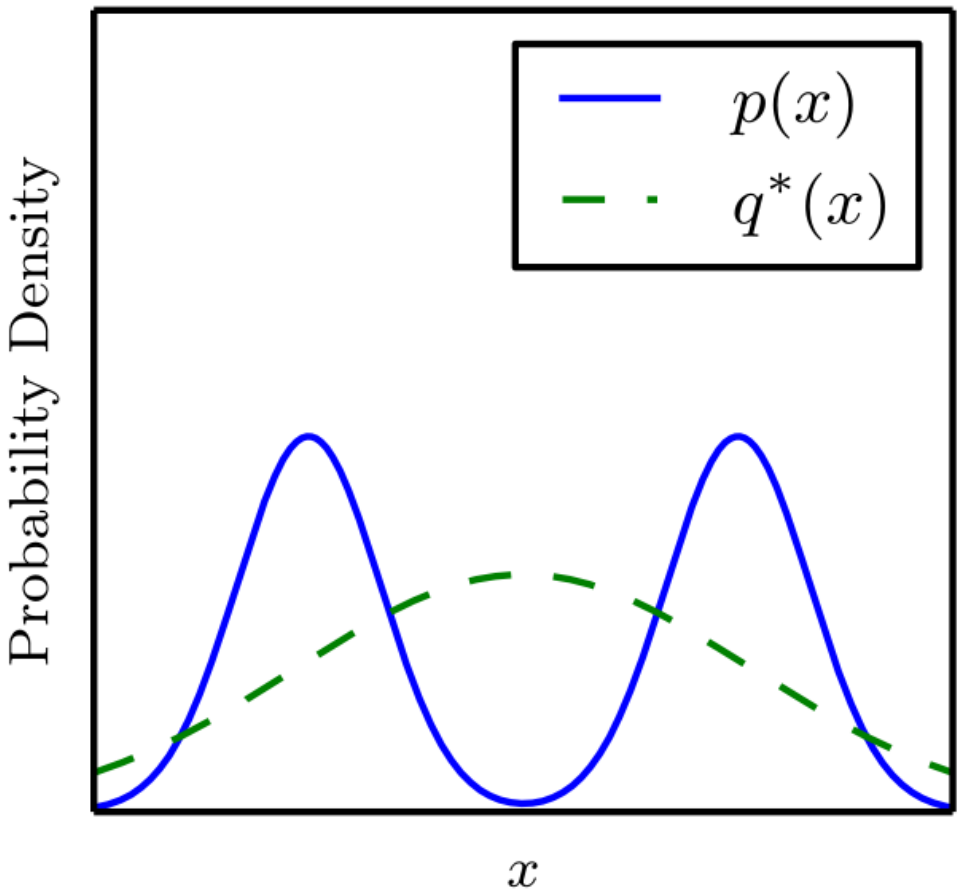
$$\begin{aligned}\mathrm{KL}(p\|q) &= -\int p(\mathbf{x})\ln q(\mathbf{x})\,\mathrm{d}\mathbf{x} - \left(-\int p(\mathbf{x})\ln p(\mathbf{x})\,\mathrm{d}\mathbf{x}\right) \\ &= -\int p(\mathbf{x})\ln\left\{\frac{q(\mathbf{x})}{p(\mathbf{x})}\right\}\mathrm{d}\mathbf{x}.\end{aligned}$$

- Is the Kullback-Leibler (KL) Divergence

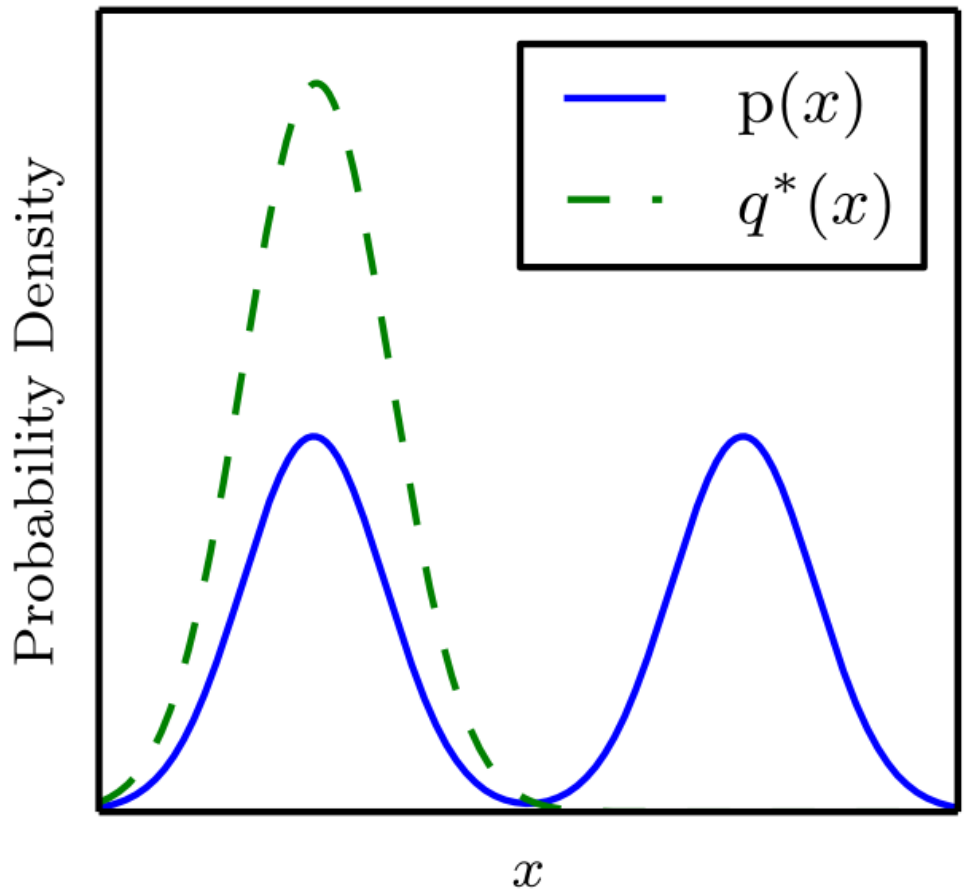
Information theory

- Kullback-Leibler (KL) Divergence is asymmetric

$$q^* = \mathrm{argmin}_q D_{\mathrm{KL}}(p\|q)$$



$$q^* = \mathrm{argmin}_q D_{\mathrm{KL}}(q\|p)$$



Gradient-based optimizaiton

- Optimization refers to either minimizing or maximizing some function $f(x)$ by altering x
  - $f(x)$ is called the objective function or criterion when we maximize
  - If we minimize is called the cost/loss/error function

$$x^* = \arg\min f(x)$$

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

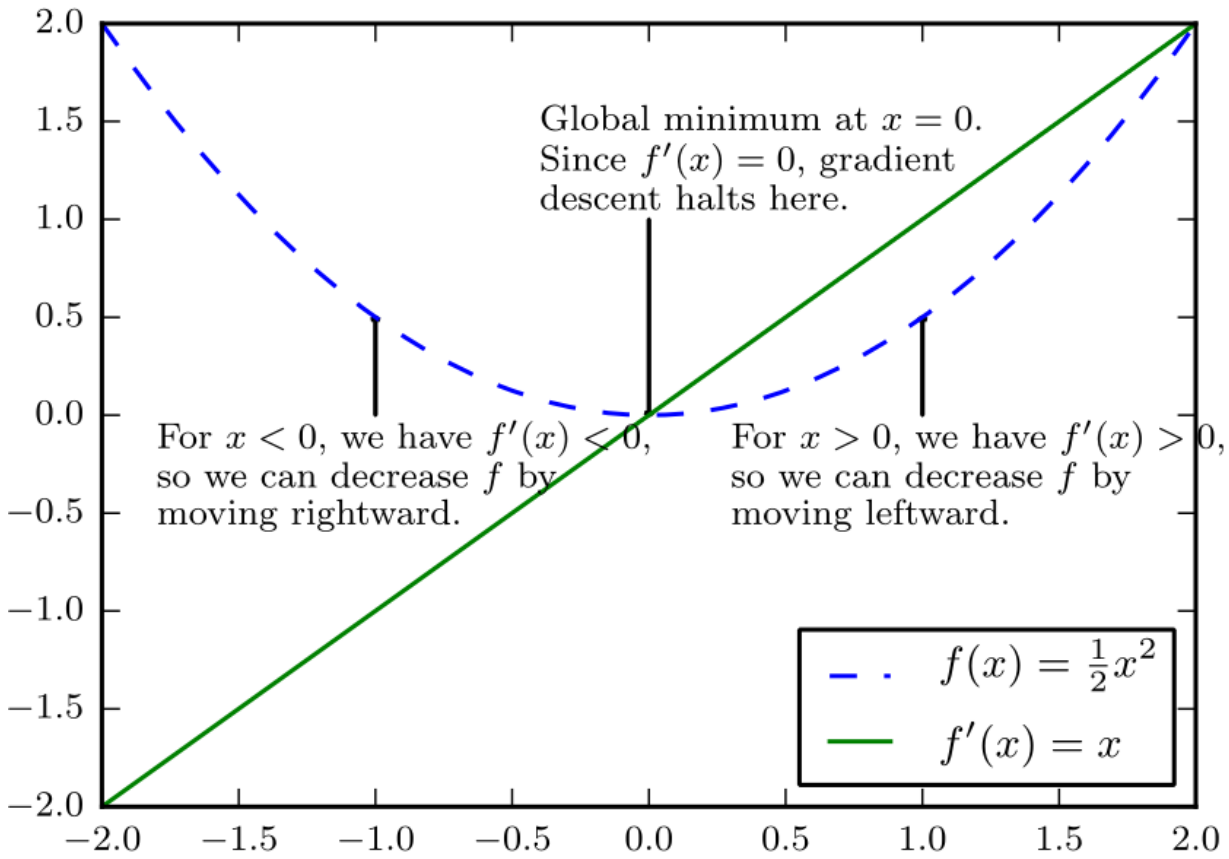
Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

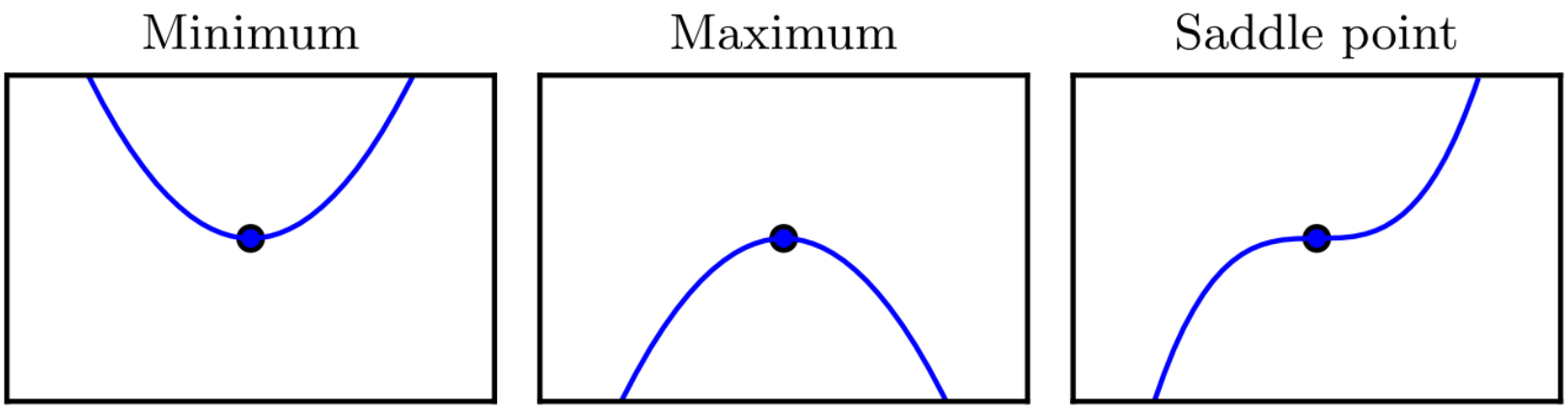
Gradient-based optimizaiton

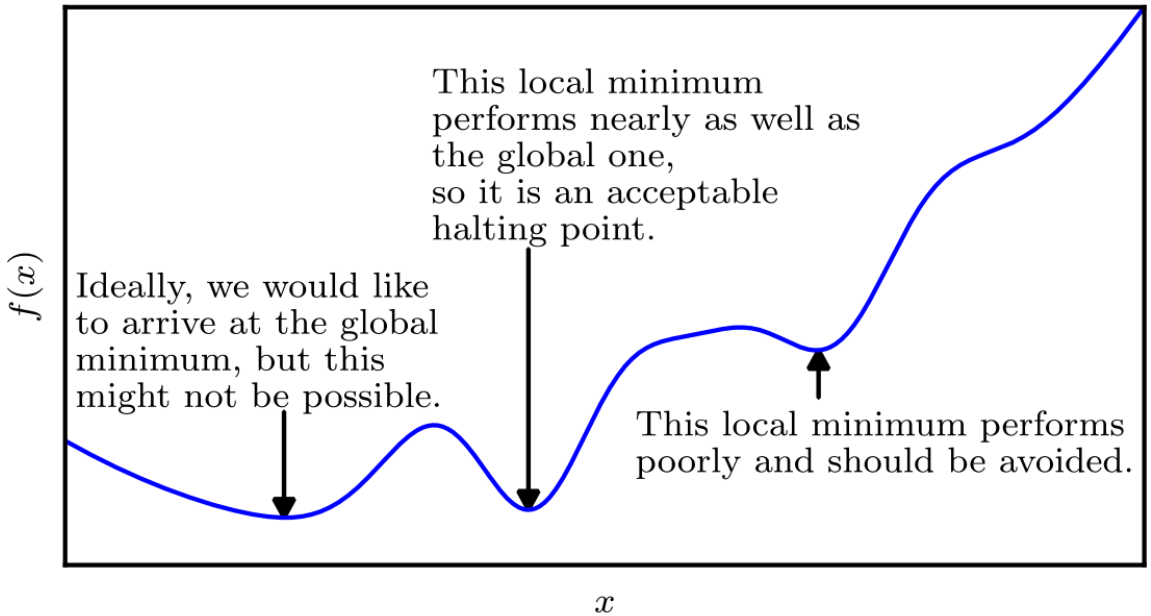
- Gradient descent



Gradient-based optimizaiton

- Gradient descent





Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

Expectation

- The average value of some function $f(x)$ under a probability distribution $p(x)$ is the expectation of $f(x)$

$$\mathbb{E}[f] = \sum_x p(x) f(x) \qquad \mathbb{E}[f] = \int p(x) f(x)\, \mathrm{d}x.$$

  - Given a finite number N of points drawn from a probability distribution, then the expectation can be approximated as

$$\mathbb{E}[f] \simeq \frac{1}{N} \sum_{n=1}^{N} f(x_n).$$

  - Conditional expectation is

$$\mathbb{E}_x[f|y] = \sum_x p(x|y) f(x)$$

Covariance

- The variance of some function $f(x)$ is defined as

$$\mathrm{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right] = \mathbb{E}[f(x)^2] - \mathbb{E}[f(x)]^2.$$

  – It measures how much variability there is in $f(x)$ around its mean value

Covariance

- The variance of some function $f(x)$ is defined as

$$\mathrm{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right] = \mathbb{E}[f(x)^2] - \mathbb{E}[f(x)]^2.$$

  – It measures how much variability there is in $f(x)$ around its mean value

- For two random variables, the covariance is defined as

$$\begin{aligned}\mathrm{cov}[x, y] &= \mathbb{E}_{x,y}\left[\{x - \mathbb{E}[x]\}\{y - \mathbb{E}[y]\}\right] \\ &= \mathbb{E}_{x,y}[xy] - \mathbb{E}[x]\mathbb{E}[y]\end{aligned}$$

  – It expresses the extent to which x and y vary together

Covariance

- The variance of some function $f(x)$ is defined as

$$\mathrm{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right] = \mathbb{E}[f(x)^2] - \mathbb{E}[f(x)]^2.$$

  – It measures how much variability there is in $f(x)$ around its mean value

- For two random variables, the covariance is defined as

$$\begin{aligned}\mathrm{cov}[x, y] &= \mathbb{E}_{x,y}\left[\{x - \mathbb{E}[x]\}\{y - \mathbb{E}[y]\}\right] \\ &= \mathbb{E}_{x,y}[xy] - \mathbb{E}[x]\mathbb{E}[y]\end{aligned}$$

  – It expresses the extent to which x and y vary together
  – In vectorial form

$$\begin{aligned}\mathrm{cov}[\mathbf{x}, \mathbf{y}] &= \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\{\mathbf{x} - \mathbb{E}[\mathbf{x}]\}\{\mathbf{y}^{\mathrm{T}} - \mathbb{E}[\mathbf{y}^{\mathrm{T}}]\}\right] \\ &= \mathbb{E}_{\mathbf{x},\mathbf{y}}[\mathbf{x}\mathbf{y}^{\mathrm{T}}] - \mathbb{E}[\mathbf{x}]\mathbb{E}[\mathbf{y}^{\mathrm{T}}].\end{aligned}$$

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD

Eigendecomposition

- Many mathematical objects can be understood better by breaking them into constituent parts
- For matrices we use eigendecomposition
  - Decompose a matrix into eigenvectors and eigenvalues

Eigendecomposition

- Many mathematical objects can be understood better by breaking them into constituent parts
- For matrices we use eigendecomposition
  - Decompose a matrix into eigenvectors and eigenvalues
- An eigenvector is a nonzero vector such that multiplication by $\boldsymbol{A}$ alters only the scale of $\boldsymbol{v}$

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$$

Eigendecomposition

- Many mathematical objects can be understood better by breaking them into constituent parts
- For matrices we use eigendecomposition
  - Decompose a matrix into eigenvectors and eigenvalues
- An eigenvector is a nonzero vector such that multiplication by $\boldsymbol{A}$ alters only the scale of $\boldsymbol{v}$

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$$

  - If $\boldsymbol{v}$ is an eigenvector of $\boldsymbol{A}$, then so is any rescaled vector $s\boldsymbol{v}$. It also has the same eigenvalue

Eigendecomposition

- An eigenvector is a nonzero vector such that multiplication by $\boldsymbol{A}$ alters only the scale of $\boldsymbol{v}$

$$\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$$

- Suppose that a matrix $\boldsymbol{A}$ has n linearly independent eigenvectors
  - We can concatenate the eigenvectors to form a matrix $\boldsymbol{V}$ with one eigenvector per column
  - We can concatenate the eigenvalues to form a vector $\boldsymbol{\lambda}$
- The eigendecomposition of $\boldsymbol{A}$ is $\boldsymbol{A} = \boldsymbol{V}\operatorname{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}$

Eigendecomposition

- The eigendecomposition of $\boldsymbol{A}$ is

$$\boldsymbol{A} = \boldsymbol{V}\mathrm{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}$$

- Every real symmetric matrix can be decomposed into an expression using only real-valued eigenvectors and eigenvalues

$$\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{\top}$$

  – The eigenvalue $\boldsymbol{\Lambda}_{i,i}$ is associated with the eigenvector in column i of $\boldsymbol{Q}$

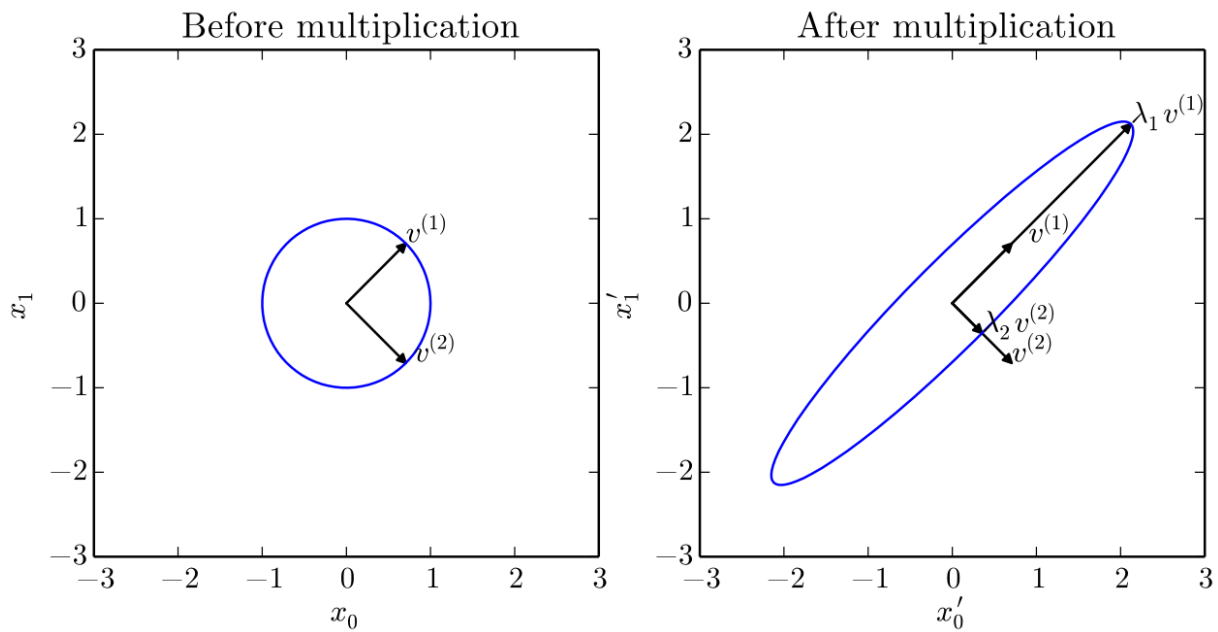
  – We can think of $\boldsymbol{A}$ as scaling space in direction $\boldsymbol{v}^{(i)}$

Eigendecomposition

- Every real symmetric matrix can be decomposed into an expression using only real-valued eigenvectors and eigenvalues

$$\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^{\top}$$

  – We can think of $\boldsymbol{A}$ as scaling space in direction $\boldsymbol{v}^{(i)}$



Eigendecomposition

- Eigendecomposition tells us facts about the matrix
  - We can think of $\boldsymbol{A}$ as scaling space in direction $\boldsymbol{v}^{(i)}$
  - The matrix is singular if and only if any of the eigenvalues are zero
  - Eigendecomposition of a real symmetric matrix can be used to optimize a quadratic expression like

    $f(x) = x^{\mathrm{T}}\boldsymbol{A}x$

    - Whenever x is equal to an eigenvector of $\boldsymbol{A}$, f takes the value of the corresponding eigenvalue
    - The maximum value of f within the constraint region is the maximum value

Eigendecomposition

- A matrix whose eigenvalues
  - are all positive is called positive definite
    - guarantee that if $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = 0 \Rightarrow \boldsymbol{x} = 0$
  - are all positive or zero-valued is called positive semidefinite
    - They guarantee that $\forall \boldsymbol{x}, \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0$

Eigendecomposition

- A matrix whose eigenvalues
  - are all positive is called positive definite
    - guarantee that if $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} = 0 \Rightarrow \boldsymbol{x} = 0$
  - are all positive or zero-valued is called positive semidefinite
    - They guarantee that $\forall \boldsymbol{x}, \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0$
  - are negative, the matrix is negative definite
  - are negative or zero-valued, it is negative semidefinite

Singular Value Decomposition

- Also to decompose a matrix into singular vector and singular values
  - Is more generally applicable, e.g. to non-square matrix
  - $\boldsymbol{A}$ is a product of three matrices

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^\top$$

    - $\boldsymbol{A}^{m \times n}$, $\boldsymbol{U}^{m \times m}$, $\boldsymbol{D}^{m \times n}$, $\boldsymbol{V}^{n \times n}$
    - $\boldsymbol{U}$ (left singular vectors) and
      $\boldsymbol{V}$ (right singular vectors) are orthogonal
    - $\boldsymbol{D}$ is diagonal containing the singular values

Review on Machine Learning

Machine Learning terminology

The curse of dimensionality

Information Theory

Gradient-Based Optimization

Expectations and Covariances

Eigendecomposition and SVD